

Q19) Write a C program for encryption in the cipher block chaining (CBC) mode using an algorithm stronger than DES. 3DES is a good candidate. Both of which follow from the definition of CBC.

Which of the two would you choose:

- a. For security?**
- b. For performance?**

Aim: To write a C program to implement **Cipher Block Chaining (CBC) mode** using the **Triple Data Encryption Standard (3DES)** algorithm for secure block encryption.

Algorithm:

1. Start the program.
2. Read the plaintext, encryption key, and Initialization Vector (IV).
3. Divide the plaintext into fixed-size blocks (8 bytes for 3DES).
4. XOR the first plaintext block with the IV.
5. Encrypt the result using the Triple DES algorithm.
6. Store the encrypted block as the ciphertext.
7. XOR each subsequent plaintext block with the previous ciphertext block.
8. Encrypt each XOR result using Triple DES.
9. Repeat until all blocks are encrypted and display the ciphertext.
10. Stop the program.

Result:

Thus, the C program to simulate **CBC mode encryption using Triple DES (3DES)** was successfully executed. It was observed that CBC mode improves security by chaining ciphertext blocks, whereas ECB mode provides better performance but weaker security due to its independent block encryption.

Answer to the Questions

a. Which mode would you choose for security?

Answer:

For **security**, **CBC (Cipher Block Chaining) mode with 3DES** is preferred.

Reasons:

- Each plaintext block is XORed with the previous ciphertext block.
- Identical plaintext blocks produce different ciphertext blocks.
- Prevents pattern leakage.
- More secure than ECB mode.
- 3DES applies DES encryption three times, making it significantly stronger than single DES.

b. Which mode would you choose for performance?

Answer:

For **performance**, **ECB (Electronic Codebook) mode** is faster because:

- Each block is encrypted independently.
- Blocks can be processed in parallel.
- No dependency on previous ciphertext blocks.
- Requires less computation than CBC.

However, ECB is less secure because identical plaintext blocks generate identical ciphertext blocks, revealing patterns in the data.

Code:

```
1 #include <stdio.h>
2 #include <string.h>
3 #define SIZE 100
4 int main()
5 {
6     char plaintext[SIZE], ciphertext[SIZE];
7     char key = 'K'; // Simple key (simulation)
8     char iv = 'A'; // Initialization Vector
9     int i, len;
10    printf("Enter Plaintext: ");
11    gets(plaintext);
12    len = strlen(plaintext);
13    // CBC Encryption Simulation
14    ciphertext[0] = (plaintext[0] ^ iv) ^ key;
15    for(i = 1; i < len; i++)
16    {
17        ciphertext[i] = (plaintext[i] ^ ciphertext[i-1]) ^ key;
18    }
19    ciphertext[len] = '\0';
20    printf("\nCiphertext (Hex): ");
21    for(i = 0; i < len; i++)
22        printf("%02X ", (unsigned char)ciphertext[i]);
23    printf("\n");
24    return 0;
25 }
```

Line: 23 Col: 18 Sel: 0 Lines: 25 Length: 704 Insert Done parsing in 0.016 seconds

Output:

```
C:\Users\SAIL\Desktop\lab16.exe
Enter Plaintext: Abhay

Ciphertext (Hex): 4B 62 41 6B 59

-----
Process exited after 14.82 seconds with return value 0
Press any key to continue . . .
```